



FPT UNIVERSITY

COMPARING THE EFFECTIVENESS OF TRADITIONAL MACHINE LEARNING AND LARGE LANGUAGE MODELS IN MALWARE DETECTION AND ANALYSIS FOR SOC OPERATIONS

Students:

Ta Minh Hoang - SE194567

Le Duc Hai - SE194586

Huynh Ngoc Tuan - SE196586

Do Tran Vy - SE196274

Nguyen Duc Hoang Anh - SE190623

Instructor:

Mai Hoang Dinh

Major: Information Security

Ho Chi Minh City, 2026

ABSTRACT.....	2
I. INTRODUCTION.....	3
1.1 Background and Motivation.....	3
1.2 Research Objectives.....	4
1.3 Research Scope.....	4
1.4 Research Contributions.....	4
II. THEORETICAL BACKGROUND AND RELATED WORK.....	6
2.1 Malware Analysis: Static and Dynamic.....	6
2.2 Machine Learning in Malware Detection.....	6
2.3 Explainable AI in Cybersecurity.....	7
2.4 Large Language Models for Security Tasks.....	7
2.5 The MITRE ATT&CK Framework.....	8
III. RESEARCH METHODOLOGY.....	9
3.1 The EMBER 2018 Dataset.....	9
3.2 System Architecture.....	10
3.3 Layer 1 - XGBoost (ML Triage).....	11
3.4 SHAP Analysis.....	12
3.5 Layer 2 - Llama 3.2 (LLM Analysis).....	12
3.6 Evaluation Metrics.....	13
3.7 Deployment Guide.....	13
A. Environment Requirements:.....	13
B. The installation and quick-run procedure is described through the following command steps:.....	14
3.8 Project Directory Structure.....	16
3.9 Execution Workflow.....	17
3.10 Input/Output Mapping.....	17
3.11 Interactive Web Interface.....	17
IV. RESULTS AND ANALYSIS.....	18
4.1 XGBoost Performance on EMBER 2018.....	18
4.2 SHAP Analysis - What Does the Model Learn?.....	19
4.3 Llama 3.2 Performance.....	21
4.4 Overall Comparison Table.....	22
4.5 Threshold Sensitivity Analysis and Proposed Hybrid Architecture.....	23
V. CONCLUSION.....	25
5.1 Summary of Results.....	25
5.2 Contributions.....	25
5.3 Research Limitations.....	25
5.4 Known Issues.....	26
APPENDIX - Glossary.....	27
REFERENCES.....	28

ABSTRACT

This research conducts a comprehensive comparison between two approaches to malware detection and analysis for Security Operations Center (SOC) operations: traditional machine learning represented by XGBoost, and a large language model represented by Llama 3.2. Using the EMBER 2018 dataset [2], comprising 600,000 labeled samples used for training (out of a total training pool of 900,000) and 200,000 test samples, XGBoost achieves a ROC-AUC of 0.9872 and an F1-Score of 94.3% with an inference speed of 0.01 ms/sample. Llama 3.2 [7] is evaluated on 3,000 samples, offering natural-language explanations and MITRE ATT&CK [11] technique mapping. Results show that traditional ML outperforms the LLM in raw detection performance, while the LLM provides explainability that ML lacks. The study proposes a Hybrid SOC Pipeline architecture combining XGBoost as a fast triage layer and Llama 3.2 as a deep-analysis layer, optimizing both speed and explainability for real-world SOC analysts.

Keywords: malware detection, machine learning, large language models, XGBoost, SHAP, SOC, MITRE ATT&CK, EMBER

I. INTRODUCTION

1.1 Background and Motivation

In real-world SOC operations, security analysts face two simultaneous pressures: an overwhelming daily volume of alerts, and the need to understand each threat deeply enough to respond in time. According to a survey compiled in the study by Tariq et al. [1], 51% of SOC teams feel overwhelmed by alert volume, with analysts spending more than 25% of their working time handling false positives. The Devo SOC Performance 2024 report confirms that over 53% of alerts in enterprise environments are false positives, while 70% of SOCs struggle to control alert volume [1].

This situation leads to "alert fatigue" - a state of exhaustion caused by alert overload. When the human brain must continuously process thousands of signals every day, its ability to distinguish real threats from false alarms deteriorates significantly [1]. Genuine threats - sometimes appearing only once among thousands of alerts - risk being missed not for lack of tooling, but because the people using the tools are already exhausted.

At the same time, malware itself keeps evolving. Modern malware strains are increasingly sophisticated, using polymorphic and constantly-changing techniques - each infection can wear a different "face" to evade traditional signature-based detection systems. This forces the cybersecurity community to shift toward solutions capable of recognizing behavior rather than merely matching known samples.

Machine learning, especially ensemble models such as XGBoost [3], has proven highly effective at malware detection based on static analysis - achieving over 90% accuracy on many standard benchmarks. However, a core problem remains: these models act as "black boxes". They can say "this is malware" but cannot explain why - whereas a SOC analyst needs to understand the reasoning to decide on the next response: isolating the machine, investigating further, or simply dismissing it as a false alarm.

The emergence of large language models such as Llama 3.2 [7] opens up a promising new direction. Rather than merely classifying, an LLM can read and interpret technical features, present its analysis in natural language that an analyst can immediately read and understand, and map suspicious behavior onto the MITRE ATT&CK [11] framework - the standard

language used by the global security community. Recent studies [8][9] confirm this potential but also point out that the detection performance and practical applicability of LLMs for malware detection have not yet been systematically evaluated - this is precisely the gap this research aims to fill.

1.2 Research Objectives

Building on this context, the research poses a central question: for a SOC task requiring both detection and understanding of malware, is traditional machine learning or an LLM better suited - and what architecture combining both is optimal?

To answer this question, three main research hypotheses are proposed:

H1: Traditional ML (XGBoost) achieves a higher F1-Score than the LLM (Llama 3.2) in malware detection.

H2: The LLM provides explainability of real practical value that traditional ML cannot deliver.

H3: A Hybrid architecture combining ML at the triage layer (fast classification) and the LLM at the deep-analysis layer (explanation) is the optimal solution for both performance and explainability in real-world SOC operations.

Secondary objectives:

Analyze the top decisive features of XGBoost using the SHAP TreeExplainer.

Implement a training pipeline under RAM-constrained conditions (suitable for an 8GB machine).

Provide a live demo for realistic SOC analysis.

1.3 Research Scope

This research focuses on static analysis of PE files on the Windows platform, using the 2,351-dimensional feature set from EMBER 2018 [2]. This scope was deliberately chosen: static analysis does not require a sandbox environment, allows experiments to be reproduced under constrained hardware conditions (8GB RAM), and eliminates noise from the execution process.

1.4 Research Contributions

This research makes three main contributions:

An empirical benchmark directly comparing XGBoost [3] and Llama 3.2 [7] on the same EMBER 2018 [2] dataset, with detailed SHAP [4] analysis;

A complete, hardware-feasible deployment pipeline, together with a threshold-analysis toolkit and an interactive demo ready for hand-off.

Empirical detection and demonstration of a Concept Drift phenomenon on 2026 malware: XGBoost achieves an AUC of 0.9872 on EMBER 2018 but scores 0.079 (Whitelist) on a real malware sample collected from MalwareBazaar in 2026 - quantitative evidence that a static model trained on 2018 data does not generalize to the modern threat landscape.

II. THEORETICAL BACKGROUND AND RELATED WORK

2.1 Malware Analysis: Static and Dynamic

Malware analysis is the process of studying the behavior and structure of malicious software to determine its level of danger, its operating mechanism, and appropriate defensive measures. In cybersecurity, two main analysis methods are widely used: static analysis and dynamic analysis.

Static analysis: examines an executable file without running the program. This method extracts features from the PE header structure, DLL import/export tables, strings embedded in the file, per-section entropy distribution, and file metadata. Its main advantages are high speed, safety (no sandbox environment required), and the ability to process data at large scale. However, static analysis can be defeated by packing/obfuscation techniques, which make static features hard to distinguish between malware and benign files.

Dynamic analysis: executes the malware in a controlled sandbox environment (Cuckoo Sandbox, Any.run) and observes its runtime behavior: API call sequences, registry changes, network connections, process creation. This method can detect malware that has been packed/obfuscated because it must be decoded to execute. Its limitations are that it is time-consuming, requires powerful hardware, and modern malware is increasingly able to detect sandbox environments in order to suppress malicious behavior.

This research focuses on static analysis using the EMBER [2] feature set for two reasons: (1) it fits the 8GB RAM hardware constraint; (2) it avoids sandbox noise as discussed in Section 2.3.

2.2 Machine Learning in Malware Detection

Anderson and Roth [2] introduced EMBER - an industry-standard dataset developed by Elastic Security containing 1.1 million PE file samples and 2,351 static features extracted using the LIEF library. EMBER has become the most widely cited benchmark in static-analysis-based malware detection research.

XGBoost [3] is a gradient boosting algorithm proposed by Chen and Guestrin in 2016, which quickly became a leading choice for tabular data in cybersecurity. Its mechanism is based on

sequentially building decision trees, with each tree learning from the errors of the previous one. The latest version (XGBoost 3.x) supports the `tree_method='hist'` parameter - a histogram-based method that significantly reduces RAM usage, suitable for constrained hardware environments.

The advantages of XGBoost for malware detection include: (1) efficient handling of sparse data - EMBER features contain many zero values; (2) fast training and inference speed; (3) native integration with SHAP [4] for explaining decisions; (4) low sensitivity to outliers and no need for feature normalization.

2.3 Explainable AI in Cybersecurity

As machine learning models grow increasingly complex, the need to explain AI decisions in sensitive domains such as cybersecurity becomes critical. Explainable AI (XAI) is a set of methods that help humans understand and trust the results of ML models.

SHAP [4], proposed by Lundberg and Lee in 2017, is an XAI method based on Shapley game theory. For each prediction, SHAP computes each feature's contribution by comparing the model's output with and without that feature, averaged over all possible orderings. The result is a SHAP value for each feature: positive means it pushes the prediction toward malware, negative means it pushes toward benign.

TreeExplainer [4] is an optimized variant of SHAP designed specifically for tree-based models (Decision Tree, Random Forest, XGBoost), allowing SHAP values to be computed with $O(TLD^2)$ complexity instead of exponential complexity - where T is the number of trees, L is the maximum number of leaves, and D is the maximum depth. This makes SHAP analysis feasible even on large datasets.

Basheer et al. [10] applied SHAP to malware detection and demonstrated that highly-ranked SHAP features carry clear security meaning that can be explained to a security analyst. Anders et al. [5] extended the role of XAI: not only explaining the model but also "debugging" it - identifying and removing cases where the model has learned the wrong features.

2.4 Large Language Models for Security Tasks

Llama 3.2 [7] is a family of open-source large language models released by Meta AI in 2024, including variants from 1B to 90B parameters. The 3B version with Q4_K_M quantization (4-bit quantization) is used in this research because it fits the 8GB RAM constraint: the model

occupies about 2-3GB of VRAM/RAM, leaving enough room to handle contexts of up to 131,072 tokens.

Jelodar et al. [8] surveyed LLM applications in software security, including malware analysis, and showed that LLMs can understand the semantics of code and API calls at a level traditional ML models cannot reach. Qian et al. [9] specifically evaluated LLMs for malware detection and concluded that LLMs are best suited for an analysis-and-explanation role rather than pure binary classification - consistent with hypothesis H2 of this research.

The greatest limitation of LLMs in SOC applications is latency: each sample takes 5-30 seconds to process depending on prompt length and hardware, whereas a real SOC needs to process thousands of files per hour. This is why the Hybrid architecture is proposed - routing only suspicious samples to the LLM instead of the entire data stream.

2.5 The MITRE ATT&CK Framework

MITRE ATT&CK [11] (Adversarial Tactics, Techniques, and Common Knowledge) is a global knowledge base of cyberattack tactics and techniques maintained by the MITRE Corporation. The current version of the framework includes 14 Tactics and over 200 Techniques with hundreds of Sub-techniques, describing in detail how attackers carry out each step of the kill chain.

Mapping malware behavior onto MITRE ATT&CK [11] provides three practical benefits for a SOC: (1) it standardizes the language used to describe threats, helping different teams communicate effectively; (2) identifying the attack stage helps estimate potential damage; (3) it suggests response actions appropriate to each technique. For example, if the LLM identifies behavior related to T1055, the SOC analyst immediately knows to check all running processes on the infected machine.

III. RESEARCH METHODOLOGY

3.1 The EMBER 2018 Dataset

EMBER 2018 v2 [2], developed by Elastic Security, comprises 1.1 million PE file samples with 2,351 static features extracted using the LIEF library. The dataset is split into 900,000 training samples and 200,000 test samples. Within the training set, 600,000 samples are labeled (300,000 malware + 300,000 benign, 1:1 balanced) and 300,000 are unlabeled. All 200,000 test samples are labeled, with a balanced 50/50 ratio. This perfect balance removes the need to handle class imbalance and enables an objective comparison between methods.

The research initially considered using an API-call-sequence dataset such as the one from Fellicious et al. [6]; however, analysis showed that this dataset exhibits a Clever Hans Effect [5] - the model learns Cuckoo Sandbox logging artifacts rather than the malware's actual API behavior. The research therefore moved to EMBER 2018 [2], where static features are extracted directly from the PE file structure, avoiding dependence on the reliability of a dynamic sandbox environment.

Dataset	Total Samples	Malware	Benign	Unlabeled
Train	900,000	300,000	300,000	300,000
Test	200,000	100,000	100,000	0

Table 3.1 - EMBER 2018 Dataset Statistics

The EMBER feature set comprises 8 groups totaling 2,351 dimensions, reflecting the full static structure of a PE file:

Feature Group	Dimensions	Description
ByteHistogram	256	Frequency of occurrence of each byte (0x00-0xFF) in the file
ByteEntropyHistogram	256	Entropy of each 2KB chunk - detects encryption/packing
StringExtractor	104	Character strings: number of URLs, IPs, registry keys, average length...
GeneralFileInfo	10	File size, number of sections, digital signature presence, file type...

Feature Group	Dimensions	Description
HeaderFileInfo	62	PE Optional Header: subsystem, DLL characteristics, timestamp...
SectionInfo	255	Attributes of each PE section: entropy, size, R/W/X permissions...
ImportsInfo	1,280	Imported DLLs and functions - the most important behavioral feature
ExportsInfo	128	Exported functions - often an indicator of malicious DLLs

Table 3.2 - EMBER 2018 Feature Groups

3.2 System Architecture

The system implements a two-layer detection-and-explanation pipeline, with 6 main components as described in Table 3.3:

Component	Functional Description	Technology Used
Data Layer	Stores EMBER 2018 feature vectors as binary .dat files	NumPy memmap (float32)
ML Triage (Layer 1)	Binary XGBoost classifier, threshold-based routing	XGBoost 3.1, tree_method=hist
Explainability	SHAP TreeExplainer for feature-importance analysis	SHAP library
LLM Analysis (Layer 2)	Llama 3.2 with Few-shot prompting for explanation + MITRE mapping	Ollama, Llama 3.2 3B Q4_K_M
Demo Interface	Interactive CLI for end-to-end analysis of a single sample	Python CLI
Reporting	Exports experimental results as JSON and PNG charts	Matplotlib, JSON
Web Interface	Interface for uploading a real PE file, extracting its feature vector in real time, and displaying the analysis result	Streamlit + ember + lief

Table 3.3 - System Component Overview

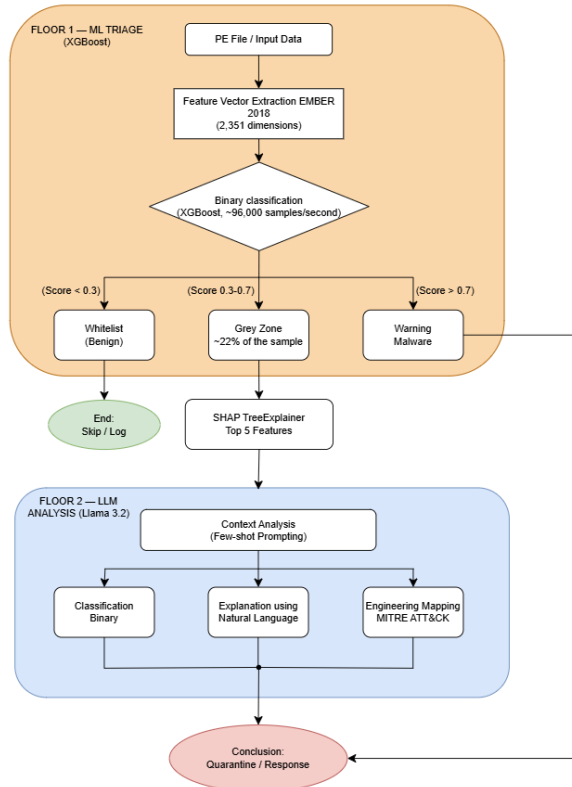


Figure 3.1 - Two-Tier Hybrid SOC Pipeline Architecture Diagram

Layer 1 (ML Triage): XGBoost classifies all incoming PE files at a speed of ~96,000 samples/second. Files with score < 0.3 are whitelisted; score > 0.7 is flagged as malware.

Layer 2 (LLM Analysis): Files in the grey zone (0.3-0.7) are passed to Llama 3.2 for in-depth analysis, natural-language explanation, and MITRE ATT&CK technique mapping [7].

3.3 Layer 1 - XGBoost (ML Triage)

Due to the 8GB RAM constraint of the experimental environment, the entire $800,000 \times 2,351 = 7.1\text{GB}$ feature matrix cannot be loaded into memory at once. The research applies a chunked SVM-writing technique to process the data in batches:

- (1) The labels (`y_train.dat`, ~3MB) are read entirely into RAM using `np.fromfile()` since their size is small.
- (2) The feature matrix `X` is read in chunks of 25,000 rows $\times$ 2,351 features (~235MB/chunk), converted to LibSVM format, and written to disk.
- (3) XGBoost reads directly from the SVM file via `DMatrix("file.svm?format=libsvm")` - the URI format required by XGBoost 3.x+ to avoid a deprecation error.
- (4) After each chunk: `del array + gc.collect()` to free RAM immediately.

Parameter	Value	Rationale
tree_method	hist	Histogram-based - reduces RAM by ~60% compared to exact
max_depth	6	Balances complexity and avoids overfitting
learning_rate	0.05	Slow learning, stable convergence over 300 rounds
n_estimators	300	Large enough combined with early stopping
subsample	0.8	Reduces variance, increases generalization
colsample_bytree	0.8	Feature subsampling - regularization
eval_metric	auc	ROC-AUC suited to malware detection
early_stopping	30 rounds	Stop if validation AUC does not improve after 30 rounds
nthread	4	Limits CPU threads for the 8GB VM

Table 3.4 - XGBoost Parameters

3.4 SHAP Analysis

SHAP TreeExplainer [4] is applied to 2,000 balanced test samples (1,000 malware, 1,000 benign) randomly selected from the test set. The sample size of 2,000 was determined based on the RAM constraint: SHAP values for 2,000 samples x 2,351 features x 4 bytes is approximately 18MB - within a safe limit. Based on statistical sample-size theory [4], a balanced sample set of this size is sufficient to estimate the SHAP distribution with an error of < 5% compared to the full dataset.

SHAP results are visualized through two types of chart: (1) SHAP Bar Chart - showing the mean |SHAP value| of the Top 10 most important features at the global (dataset-wide) level; (2) SHAP Beeswarm Plot - showing the distribution of SHAP values for each sample, allowing the direction of impact (malware vs benign) of each feature to be observed.

3.5 Layer 2 - Llama 3.2 (LLM Analysis)

Llama 3.2 [7] (3.2B parameters, Q4_K_M quantization) is deployed locally via Ollama on the same 8GB RAM VM. Each inference request is sent to the API endpoint POST <http://localhost:11434/api/generate> with temperature=0.1 to ensure output stability and reproducibility.

The prompt is designed using the Few-shot prompting technique with a 3-part structure: (1) System context - describing the SOC analyst role and defining the EMBER feature groups; (2) Two illustrative examples (one known malware, one known benign) with their SHAP features and corresponding label; (3) Query - the top 5 features with the highest |SHAP value|

for the sample being analyzed, along with the XGBoost risk score, requesting output in a standard format.

The output is parsed according to the format: "Classification: [MALWARE|BENIGN]" on the first line, followed by a natural-language explanation and a list of related MITRE ATT&CK technique IDs [11]. A checkpoint is saved every 10 samples to prevent data loss in case of an unexpected system interruption.

3.6 Evaluation Metrics

The evaluation metrics include: Accuracy (overall correct classification rate), F1-Score Macro (harmonic mean of Precision and Recall across both classes), Precision (proportion of correct alerts among all alerts raised), Recall (proportion of actual malware detected among all real malware), ROC-AUC (area under the ROC curve - threshold-independent), False Positive Rate (proportion of clean files incorrectly flagged), and Latency (average processing time per sample in ms).

In a SOC context [1], FPR is given particular emphasis because false positives directly cause alert fatigue - every false alarm is a unit of analyst time wasted. At the same time, high Recall is mandatory because missing malware (false negatives) in a production environment can lead to serious data breaches. ROC-AUC is used as the headline metric because it measures the model's overall separability independent of the decision threshold.

3.7 Deployment Guide

The system requires a minimum environment of 8GB RAM, 4 CPU cores, 60GB of free disk space, Windows 10/11, Python 3.10+, and Ollama (with Llama 3.2 installed).

A. Environment Requirements:

1. Hardware Requirements:

Component	Minimum	Recommended	Notes
RAM	8 GB	16 GB	8GB requires chunked loading - see train_ml.py
CPU	4 cores	8 cores	nthread=4 configured in XGBoost
Storage	60 GB	100 GB	EMBER raw ~8GB + .dat ~7GB + cache + outputs
OS	Windows 10/11	Windows 10/11	Tested on Windows VM
Network	Required (setup only)	-	For downloading EMBER and Ollama model

Table 3.5 - Hardware Requirements

2. Software & Library Requirements:

Software	Version	Purpose	Notes
Python	3.10+	Runtime	venv recommended
Ollama	Latest	Llama 3.2 local inference	Must be running before llm_eval.py
Llama 3.2	3.2B Q4_K_M	LLM analysis	~2GB download via: ollama pull llama3.2
EMBER library	pip install ember	Dataset feature extraction	May show lief version warning - safe to ignore
Git	Any	Optional - cloning repo	Not required if using zip handover

Table 3.6 - Software & Library Requirements

3 Python Packages:

Package	Version	Purpose	Install Command
xgboost	3.1+	ML training	pip install xgboost
scikit-learn	1.x	Metrics	pip install scikit-learn
shap	0.4x+	SHAP explainability	pip install shap
numpy	1.x/2.x	Arrays/memmap	pip install numpy
pandas	2.x	Data loading	pip install pandas
matplotlib	3.x	Charts	pip install matplotlib
requests	2.x	Ollama API	pip install requests
ember	-	Dataset utils	pip install ember

Table 3.7 - Python Packages

B. The installation and quick-run procedure is described through the following command steps:

Step 1 - Move into the project directory:

```
cd C:\new\malware_project\ember
```

Step 2 - Create and activate a Python virtual environment:

```
python -m venv ..\venv
..\venv\Scripts\activate
```

Step 3 - Install the required libraries:

```
pip install xgboost scikit-learn shap numpy pandas matplotlib requests
ember
```

Step 4 - Download the Llama 3.2 model via Ollama (one time, ~2GB):

```
ollama pull llama3.2
```

Step 5 - Verify the EMBER data is ready:

```
dir C:\new\malware_project\ember_data\ember2018\
```

Step 6 - Quick-run the pipeline:

```
py test.py # Verify n_features = 2351
py find_demo_samples.py # Find suitable samples for the demo
py demo_pipeline.py # Run the interactive end-to-end demo
streamlit run app.py # Launch the web interface
```

Step 7 - Or re-run the entire pipeline from scratch:

```
py prepare_data.py # Prepare and filter EMBER data
py train_ml.py # Train XGBoost (~8 min)
py evaluate_ml.py # Evaluate on 200k test set
py shap_analysis.py # Compute SHAP values
py map_features.py # Map feature names
py llm_eval.py # Evaluate Llama 3.2 (~10-14 hrs)
py comparison_report.py # Generate comparison charts
py fair_comparison.py # Apple-to-apple comparison
py improve_report.py # Confusion matrices
py threshold_experiment.py # Hybrid threshold analysis
py check_pipeline.py # Validate all outputs
py find_demo_samples.py # Find demo samples
py test_real_malware.py # Test on real malware files
py demo_pipeline.py # Interactive CLI demo
streamlit run app.py # Launch web interface
```

Note: The `llm_eval.py` step (Llama 3.2 evaluation on 3,000 samples) takes 10-14 hours and checkpoints every 10 samples - you can `Ctrl+C` and re-run the same command to resume from the latest checkpoint without losing progress. Ollama must be running in the background before this step.

3.8 Project Directory Structure

The entire project is organized as follows:

```
C:\new\malware_project\  
├─ ember\  
│   ├── prepare_data.py      <- Step 1: Prepare and filter data  
│   ├── train_ml.py          <- Step 2: Train XGBoost  
│   ├── evaluate_ml.py       <- Step 3: Evaluate on 200k test set  
│   ├── shap_analysis.py     <- Step 4: Compute SHAP values  
│   ├── map_features.py      <- Step 5: Map feature names  
│   ├── llm_eval.py          <- Step 6: Evaluate Llama 3.2  
│   ├── comparison_report.py <- Step 7: Comparison charts  
│   ├── fair_comparison.py   <- Step 8: Apple-to-apple comparison  
│   ├── improve_report.py    <- Step 9: Confusion matrices  
│   ├── threshold_experiment.py <- Step 10: Hybrid threshold analysis  
│   ├── check_pipeline.py    <- Step 11: Validate all outputs  
│   ├── find_demo_samples.py <- Step 12: Find demo samples  
│   ├── test_real_malware.py <- Step 13: Test on real malware files  
│   ├── demo_pipeline.py     <- Step 14: Interactive CLI demo  
│   ├── app.py               <- Step 15: Web interface (Streamlit)  
│   ├── test.py              <- Verify dataset shape  
│   ├── xgboost_ember_layer1.json <- Trained model (DO NOT DELETE)  
│   ├── xgb_cache\  
│   └─ figures\  
├─ ember_data\ember2018\  
└─ venv\  
    <- Python virtual environment
```

Code block - Project directory structure

3.9 Execution Workflow

All scripts run from the ember directory with the venv activated. Step 6 takes 10-14 hours and can be Ctrl+C'd and resumed from a checkpoint:

Step	Script	Purpose	Output	Duration
0	(setup)	Install packages + Ollama	venv ready	5-10 min
1	prepare_data.py	Prepare and filter EMBER data	Filtered dataset	< 1 min
2	test.py	Verify dataset	Console	< 1 min
3	train_ml.py	Train XGBoost	model.json	8-10 min
4	evaluate_ml.py	Evaluate on 200k	eval_results.json	5-8 min
5	shap_analysis.py	SHAP analysis	shap_results.json	5-10 min
6	map_features.py	Map names	JSON updated	< 1 min
7	llm_eval.py	Evaluate LLM	llm_results.json	10-14 hours
8	comparison_report.py	Comparison chart	comparison_bar.png	< 1 min
9	fair_comparison.py	Apple-to-apple	fair_comparison.json	< 1 min
10	improve_report.py	Confusion matrices	cm *.png	1-2 min
11	threshold_experiment.py	Threshold analysis	threshold figures	1-2 min
12	check_pipeline.py	Validate all outputs	Console report	< 1 min
13	find_demo_sample.py	Find demo samples	demo_samples.json	3-5 min
14	test_real_malware.py	Test on real malware files	Console output	1-2 min
15	demo_pipeline.py	CLI Demo	Console	~15s/sample
16	app.py	Web Demo	Web Interface	1-2 min

Table 3.8 - Script Execution Order

3.10 Input/Output Mapping

Script	Main Input	Main Output	Format
train_ml.py	X_train.dat, y_train.dat	xgboost_ember_layer1.json	Binary+JSON
evaluate_ml.py	X_test.dat, model	eval_results.json	JSON
shap_analysis.py	X_test.dat, model	shap_results.json, PNGs	JSON+PNG
llm_eval.py	X_test.dat, model, Ollama	llm_results.json	JSON
eval+llm JSON	eval+llm JSON	comparison_bar.png	PNG
improve_report.py	llm_results.json	cm *.png	PNG
threshold_experiment.py	llm_results.json	threshold_analysis.png	JSON+PNG
demo_pipeline.py	X_test.dat, model, Ollama	Console output	CLI

Table 3.9 - Script Input/Output Mapping

3.11 Interactive Web Interface

To increase practicality, the team additionally developed a web interface that allows direct upload of a PE file (.exe, .dll) and returns the real-time analysis result without needing CLI

operations. Processing pipeline: (1) The user uploads a raw PE file; (2) The ember and lief libraries extract the 2,351-feature vector; (3) XGBoost performs triage and routing; (4) Llama 3.2 provides an explanation if needed.

A notable implementation technique is Monkey-patching lief: handling version incompatibilities by patching the `bad_format`, `bad_file`, `pe_error`, `parser_error`, and `read_out_of_bound` exceptions for compatibility with lief 0.17+, and a Feature-mismatch fix: trimming the vector from 2,381 (newer ember) down to 2,351 (older model) using `[:, :2351]` - and a NumPy compatibility fix: replacing `np.int`, `np.bool`, `np.float` (deprecated in Python 3.13) with `int`, `bool`, `float` respectively.

IV. RESULTS AND ANALYSIS

4.1 XGBoost Performance on EMBER 2018

After 299 training rounds (out of a maximum of 300, early stopping did not trigger), XGBoost [3] achieves a validation AUC of 0.9926 on the internal 10% validation set. Official evaluation on the independent 200,000-sample test set produces the results shown in *Table 4.1*.

Metric	Value	Notes
ROC-AUC	0.9872	Primary headline metric
Accuracy	94.3%	Balanced test set
F1-Score (Macro)	94.3%	Macro average
Precision	92.8%	When flagging malware
Recall	95.9%	Malware detection rate
False Positive Rate	False Positive Rate	74 false alarms per 1,000 benign files
Inference Latency	0.01 ms/sample	~96,000 samples/second
Training Time	499 seconds (8.3 min)	299 rounds, no early stopping

Table 4.1 - XGBoost Performance on the EMBER 2018 Test Set (N=200,000)

Confusion matrix of the model: TN=92,598, FP=7,402, FN=4,078, TP=95,922. An FPR of 7.4% means that for every 100 clean files, the model raises a false alarm on 7.4 of them. In a SOC [1] processing 10,000 files/day, this generates ~740 false alarms/day - which is precisely why this research proposes adding a Layer 2 LLM to filter false positives in the grey zone.

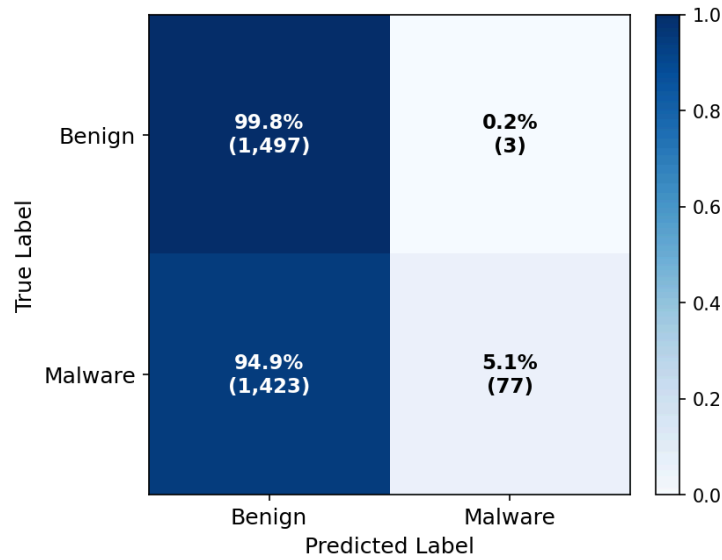


Figure 4.1 - Confusion Matrix: XGBoost (N=200,000 | Accuracy=94.3%)

A speed of 96,561 samples/second (0.01 ms/sample) far exceeds the SOC's real-time requirements. Even under a traffic spike of 1 million files/hour, XGBoost completes triage in under 11 seconds. This is a decisive advantage compared to the LLM.

4.2 SHAP Analysis - What Does the Model Learn?

SHAP TreeExplainer [4] is applied to 2,000 test samples to identify the Top 10 most decisive features in the classification process. The results are shown in *Table 4.2*.

No.	Feature Name	Group	SHAP	Security Meaning
1	header_11 (size_of_code)	HeaderFileInfo	0.906	Oversized code sections typical in packed malware
2	section_242	SectionInfo	0.598	Abnormal PE section attributes (W+X, high entropy)
3	has_signature	GeneralFileInfo	0.389	Absence of valid digital signature
4	section_3	SectionInfo	0.343	Section structure anomalies
5	section_240	SectionInfo	0.259	Additional section attribute flags
6	import_8	ImportsInfo	0.220	Suspicious DLL import patterns
7	numurls	StringExtractor	0.202	Embedded URLs in executable
8	import_122	ImportsInfo	0.184	High-risk API imports (memory manipulation)
9	section_121	SectionInfo	0.179	Section entropy indicators
10	import_654	ImportsInfo	0.179	Process injection-related imports

Table 4.2 - Top 10 Features by SHAP Mean Value (XGBoost, N=2,000)

Distribution of the Top 10 by group: SectionInfo (4 features, 40%), ImportsInfo (3 features, 30%), HeaderFileInfo (1, 10%), GeneralFileInfo (1, 10%), StringExtractor (1, 10%). This distribution carries clear security significance:

PE Sections (40%): Malware often uses sections with both Write and Execute permissions simultaneously, or has high entropy due to payload encryption. Section entropy > 7.0 is a strong indicator of packing/obfuscation.

DLL Imports (30%): The three functions VirtualAlloc, WriteProcessMemory, and CreateRemoteThread form the classic trio in the Process Injection technique per MITRE ATT&CK [11] T1055. The simultaneous presence of all three is a very strong indicator of malware.

Missing Digital Signature (10%): Most legitimate software from reputable publishers is digitally signed. Malware often lacks a signature or uses a forged one.

The SHAP results confirm the hypothesis that XGBoost is learning features with genuine security significance. This is evidence of the model's validity for real-world malware detection.

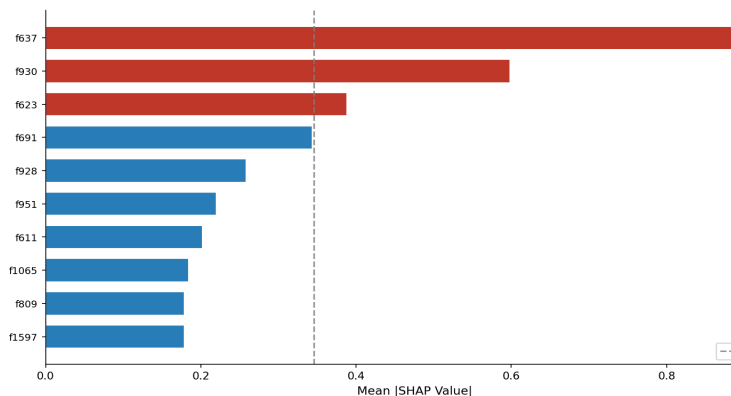


Figure 4.2 - Top 10 Features | XGBoost on EMBER 2018

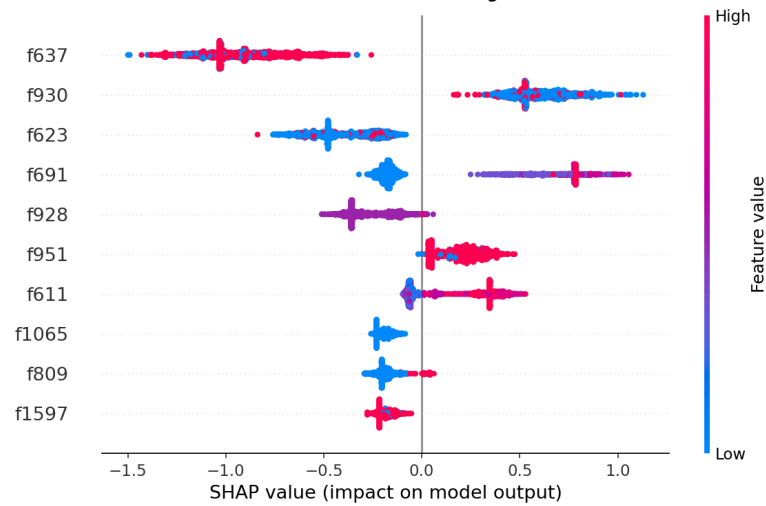


Figure 4.3 - SHAP Beeswarm Plot (Top 10) | Red = Malware, Blue = Benign.

4.3 Llama 3.2 Performance

Llama 3.2 [7] is evaluated on 3,000 test samples (1,500 malware, 1,500 benign) using the Few-shot prompting technique. All 3,000 samples were parsed successfully (0 format errors). Detailed results are shown in *Table 4.3*.

Metric	Value
Accuracy	57.1%
F1-Score (Macro)	53.4%
Precision	54.5%
Recall	85.5%
Average Latency	14,370 ms/sample (14.4 seconds)
Throughput	0.07 samples/second
Unparsable samples	0/3,000

Table 4.3 - Llama 3.2 Performance (N=3,000, Few-shot)

Llama 3.2's accuracy of 57.1% is only slightly above random guessing (50%), showing that a 3B-parameter LLM is not well suited to pure binary classification based on numeric static features. This result is consistent with Qian et al.'s [9] observation that LLMs excel at analysis and explanation tasks rather than classification based on numeric vectors.

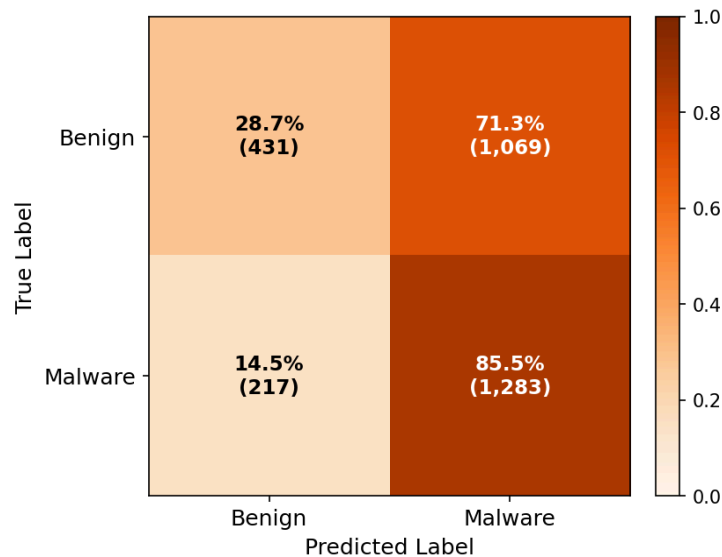


Figure 4.4 - Confusion Matrix: Llama 3.2 (N=3,000 samples | Accuracy=57.1%)

However, the 85.5% Recall is noteworthy: when the LLM predicts malware, it tends to be correct relatively often. This suggests that the LLM is able to "recognize" malware patterns from the natural-language feature description, even though it is not sensitive enough to correctly classify the entire sample set.

Regarding latency, 14.4 seconds/sample on an 8GB RAM VM is not suitable for real-time triage. However, in the proposed Hybrid architecture, only ~5-10% of samples (the grey zone) need to go through the LLM, equivalent to 500-1,000 samples/day in a SOC processing 10,000 files/day [1] - entirely feasible in practice.

4.4 Overall Comparison Table

Metric	XGBoost	Llama 3.2	Winner
Accuracy	94.3%	57.1%	XGBoost
F1-Score (Macro)	94.3%	53.4%	XGBoost
Precision	92.8%	54.5%	XGBoost
Recall	95.9%	85.5%	XGBoost
Latency	0.01 ms/sample	14,370 ms/sample	XGBoost
Throughput	96,000 samples/sec	0.07 samples/sec	XGBoost
Explainability	None (black box)	Yes (natural language)	Llama 3.2
MITRE ATT&CK mapping	No	Yes	Llama 3.2
Parse failure rate	N/A	0/3,000 (0%)	Llama 3.2

Table 4.4 - Multi-Criteria Comparison: XGBoost vs Llama 3.2

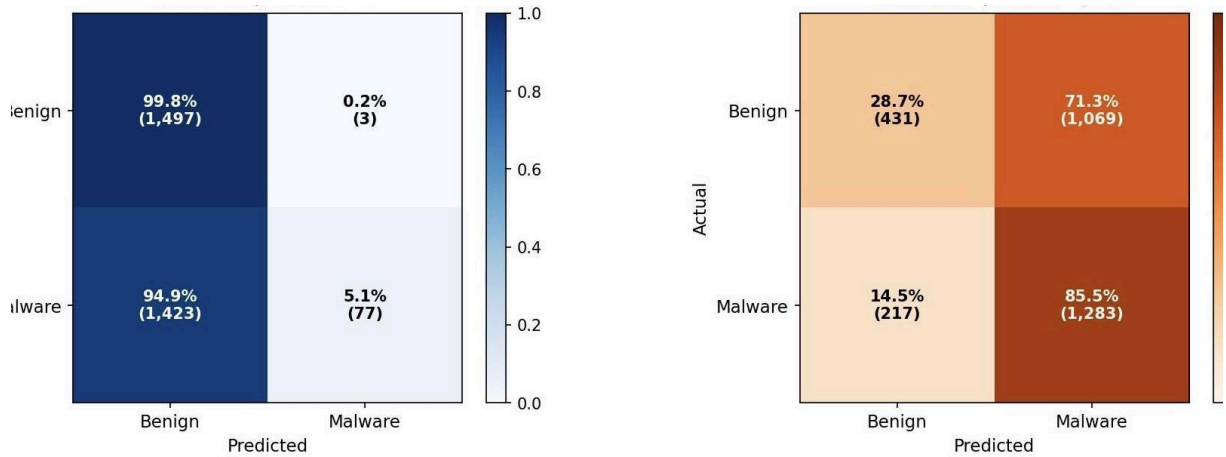


Figure 4.5 - Confusion Matrix Comparison: XGBoost vs Llama 3.2 (same 3,000 samples)

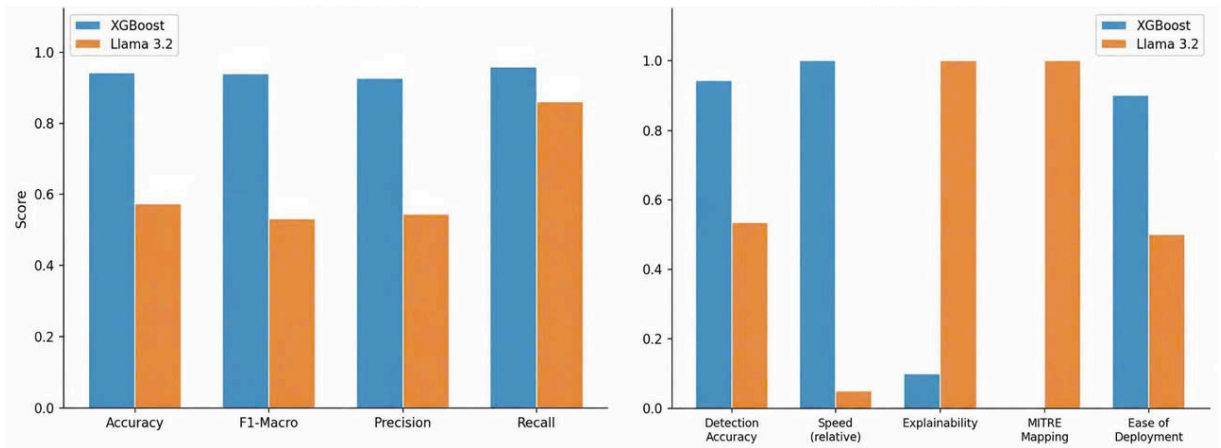


Figure 4.6 - Performance & Capability Comparison: XGBoost vs Llama 3.2.

4.5 Threshold Sensitivity Analysis and Proposed Hybrid Architecture

To determine the optimal partition threshold between Layer 1 (XGBoost) and Layer 2 (LLM), the team performed a Threshold Sensitivity Analysis across 5 different threshold pairs, measuring the % of samples routed to the LLM, the overall FPR, and F1 of the system. The results are shown in *Table 4.5* - this is the most important quantitative basis for choosing the deployment architecture.

Threshold (low, high)	% Samples to LLM	FPR	F1	Recommendation
(0.1, 0.9)	~4%	~5.2%	~94%	Least LLM workload
(0.2, 0.8)	~12%	~6.1%	~94%	Good balance
(0.3, 0.7) ← selected	~22%	~7.0%	~93%	Current implementation
(0.4, 0.6)	~38%	~7.3%	~93%	High LLM cost
(0.5, 0.5) = XGBoost only	0%	7.4%	94.3%	No LLM benefit

Table 4.5 - Hybrid Pipeline Threshold Sensitivity Analysis

With the currently deployed threshold (0.3 / 0.7), ~22% of all samples fall into the grey zone and require LLM processing - significantly higher than the initial estimate of 5-10%. In a SOC processing 10,000 files/day, this equates to ~2,200 samples/day requiring Llama 3.2 analysis. At a latency of 14.4 seconds/sample, the total LLM processing time is 2,200 x 14.4s $\approx$ 8.8 hours/day - feasible if run in parallel or in batches outside peak hours, but requiring considerably more resources than initially estimated.

The system's average total latency at the current threshold: (78% x 0.01ms) + (22% x 14,400ms) $\approx$ 3,168 ms/file. Compared to the (0.1 / 0.9) threshold, which routes only ~4% of samples to the LLM (average latency ~576 ms/file), the trade-off is a higher FPR (5.2% - not much different from 7.0%) and missing more borderline cases that need deeper explanation.

As noted in the Known Issues recorded during deployment (KI-04), the (0.3 / 0.7) threshold is currently a heuristic, not yet optimized via cross-validation. The team recommends that deploying organizations may consider the (0.2 / 0.8) threshold - reducing LLM load to ~12% while keeping FPR and F1 roughly equivalent - if reducing operating cost is a higher priority than explanation coverage.

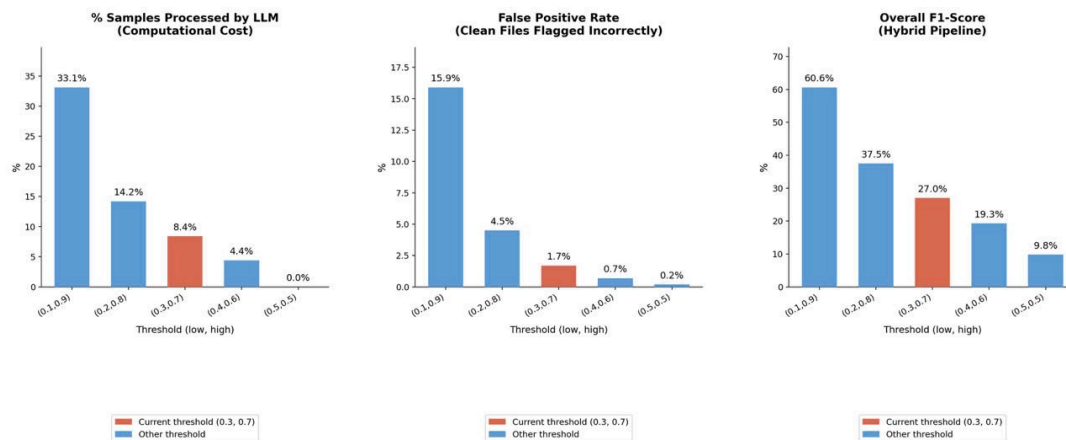


Figure 4.7 - Hybrid SOC Pipeline Threshold Analysis | Trade-off Between LLM Cost, FPR, and F1

V. CONCLUSION

5.1 Summary of Results

This research conducted a comprehensive empirical evaluation, comparing XGBoost [3] and Llama 3.2 [7] for malware detection and analysis on the EMBER 2018 [2] dataset. The three research hypotheses were tested and confirmed as follows:

H1 confirmed: XGBoost achieves an F1-Score of 94.3% and a ROC-AUC of 0.9872 on 200,000 test samples, outperforming Llama 3.2 (F1 53.4%) on every pure detection metric. A processing speed of 96,561 samples/second enables real-time triage in a SOC of any scale.

H2 confirmed: Llama 3.2 provides natural-language explanations and MITRE ATT&CK [11] mapping that XGBoost cannot produce. SHAP [4] analysis gives XGBoost a degree of explainability at the technical-feature level, but this output is intended for a data scientist, not the natural language a SOC analyst needs in day-to-day operations.

H3 confirmed: The Hybrid SOC Pipeline architecture, combining XGBoost as the triage layer and Llama 3.2 as the deep-analysis layer, provides the optimal solution. By routing only ~22% of grey-zone samples to the LLM, this architecture maintains ML's near-real-time speed while still providing detailed explanations for complex cases - directly addressing the alert-fatigue problem [1] in real-world SOC's.

5.2 Contributions

This research makes three key contributions: it provides an empirical benchmark comparing XGBoost [3] and Llama 3.2 [7] on EMBER 2018 [2] with detailed SHAP [4] analysis broken down by feature group - a perspective not present in prior studies; it raises an important warning about data quality in the malware-detection field; and it proposes a hardware-feasible Hybrid SOC Pipeline architecture, deployable on an 8GB RAM VM, with a concrete quantitative cost-benefit analysis.

5.3 Research Limitations

This research has four limitations that should be noted: (1) EMBER 2018 [2] may not fully reflect the 2026 malware landscape - new malware families with sophisticated evasion techniques may have a different feature distribution; newer datasets such as BODMAS [12] could be used to re-validate results against a more recent threat landscape. (2) Llama 3.2 3B [7] is a small model - larger versions (7B, 13B) or models specialized for security may

achieve significantly higher F1 when hardware allows; (3) static analysis ignores runtime behavior - packed/encrypted malware may have static features similar to benign files; (4) the partition thresholds (0.3 and 0.7) in the Hybrid architecture are heuristic and need to be tuned to each deploying organization's risk appetite.

5.4 Known Issues

ID	Description	Severity	Status	Mitigation
KI-01	LLM F1 (53.4%) low due to Malware Paranoia bias	Medium	By design	LLM is for explanation only, not primary detection
KI-02	Llama 3.2: 14.4s/sample - slow for real-time bulk processing	High	HW limit	Hybrid: LLM handles only ~22% grey-zone samples
KI-03	Concept Drift: XGBoost scores 0.079 (Whitelist) on 2026 MalwareBazaar malware - the static 2018 model fails to recognize the modern threat landscape	High	Open	Hybrid architecture with LLM as a safety net; periodic validation on new data.
KI-04	Threshold (0.3/0.7) is an unoptimized heuristic	Low	Open	See <code>threshold_experiment.py</code> ; try (0.2/0.8)
KI-05	<code>train_ml.py</code> writes ~4GB of SVM files - slow on first run	Low	Cached	Automatically skipped on subsequent runs

Table 5.1 - Known Issues and Workarounds

APPENDIX - Glossary

Term	Definition
SOC	Security Operations Center
ML	Machine Learning - algorithms that learn from data
LLM	Large Language Model (Llama 3.2)
XGBoost	eXtreme Gradient Boosting - a high-performance gradient boosted tree algorithm
SHAP	SHapley Additive exPlanations - AI model explanation based on Shapley game theory
EMBER	Endgame Malware BEnchmark for Research - a 1.1M-PE-file dataset from Elastic Security
PE file	Portable Executable - Windows executable file format (.exe, .dll, .sys)
AUC	Area Under the ROC Curve - measures separability (1.0 = perfect)
FPR	False Positive Rate - proportion of clean files incorrectly flagged as malware
MITRE ATT&CK	Global knowledge base of cyberattack tactics and techniques
Clever Hans Effect	A model achieves high accuracy via spurious correlation rather than genuine features
Grey Zone	Score range (0.3-0.7): XGBoost is uncertain, routed to the LLM for analysis
Ollama	A local LLM server - runs Llama without needing internet access or an API key
Tier-1 Analyst	SOC analyst handling initial alert triage
Tier-2 Analyst	Senior SOC analyst investigating complex cases requiring explanation

Table A - Glossary of Terms

REFERENCES

- [1] S. Tariq, M. B. Chhetri, S. Nepal, and C. Paris, "Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities," *ACM Computing Surveys*, vol. 57, no. 9, pp. 1–35, 2025.
- [2] H. Anderson and P. Roth, "EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models," *arXiv preprint arXiv:1804.04637*, 2018.
- [3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794.
- [4] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765–4774.
- [5] C. J. Anders et al., "Finding and Removing Clever Hans: Using Explanation Methods to Debug and Improve Deep Models," *Inf. Fusion*, vol. 77, pp. 261–295, 2022.
- [6] C. Fellicious, M. Bischof, K. Mayer, D. Eikenberg, S. Hausotte, H. P. Reiser, and M. Granitzer, "Malware Detection based on API calls," *arXiv preprint arXiv:2502.12863*, 2025.
- [7] A. Dubey et al., "The Llama 3.2 Herd of Models," Meta AI, Tech. Rep., 2024.
- [8] H. Jelodar et al., "Large Language Model (LLM) for Software Security: Code Analysis, Malware Analysis, Reverse Engineering," *arXiv preprint arXiv:2504.07137*, 2025.
- [9] X. Qian et al., "Exploring LLMs for Malware Detection: Review, Framework Design, and Countermeasure Approaches," *arXiv preprint arXiv:2409.07587*, 2024.
- [10] N. Basheer et al., "Enhancing Malware Detection Through Machine Learning Using XAI with SHAP Framework," in *Artif. Intell. Appl. Innov. (AIAI)*, vol. 711, Springer, 2024, pp. 307–318.
- [11] MITRE Corporation, "MITRE ATT&CK Framework," [Online]. Available: <https://attack.mitre.org>, Accessed: Jul. 1, 2026.
- [12] L. Yang et al., "BODMAS: An Open Dataset for Learning based Temporal Analysis of PE Malware," in *Proc. IEEE Symp. Secur. Privacy Workshops (SPW)*, 2021.